MPOO 2027-1 · GRUPO 6 · PRÁCTICA 10 · MIÉRCOLES 4 DE NOVIEMBRE

La mazmorra, parte 2

Polimorfismo, primera parte. Las herramientas dejan de ser todas iguales: cada una grita y golpea a su manera, y el juego las usa sin preguntar cuál es cuál.

Descargar los archivos (.zip)

Qué vas a hacer

En pareja (o solo), sobre **tu código de la P9**. Si no la terminaste, el .zip trae una base lista:

1. **De dónde venimos y qué es el polimorfismo**
   15 min · la P9 en una tabla, la idea de hoy y el diagrama
2. **Parte A, ejemplo guiado**
   30 min · `Herramienta` abstracta, el `Arco`, el personaje con una herramienta y el punto de control
3. **Parte B, por tu cuenta**
   20 min · la `Espada` y el `Baston`, con el mismo patrón que el arco
4. **Parte C, sin y con polimorfismo**
   20 min · el mismo recorrido con `instanceof` y sin él
5. **Parte D, los moldes**
   10 min · upcast, `instanceof` y downcast

Lo que no termines en clase lo terminas en casa. La entrega es el **viernes 6 de noviembre, 23:59**, en `p10/`.

Esta vez sin JUnit

Tus pruebas de la P9 hacen `new Herramienta(...)`, y hoy `Herramienta` se vuelve abstracta: ya no compilan, y está bien. **No las borres; solo no las compiles**: todos los comandos de hoy son sobre `src/`.

De dónde venimos: la P9 en una tabla

La P9 fue **La mazmorra**: un videojuego de texto con personajes que cargan herramientas y pelean. Si no la hiciste, el .zip trae `base-p09/` con las siete clases terminadas; léelas antes de empezar.

DE LA P9	QUÉ ERA	HOY
`Personaje` abstracta; `Caballero`, `Mago`, `Arquero`	Herencia: cada tipo tiene su vida y su `calcularDanio`	Las hijas no cambian; `Personaje` carga una herramienta en vez del inventario
`Herramienta`	Una sola clase: nombre, daño y durabilidad; `usar()` desgasta y devuelve el daño	**Se vuelve abstracta** y le salen tres hijas
`Inventario`	Composición: hasta tres herramientas por personaje; `usarMejor()` usa la sana que más pega	**Desaparece**: cada personaje carga **una** herramienta, la de su tipo (el caballero la espada, el mago el bastón, el arquero el arco)
`companero`, encapsulamiento	Asociación entre personajes; atributos privados, trampas que no compilan	No cambian

En la P9 ya usaste `abstract`, `extends` y `@Override`. Hoy ves **para qué sirven**: para que el juego dé una orden y cada objeto responda a su manera. Eso se llama **polimorfismo**.

Qué es el polimorfismo

Polimorfismo quiere decir «muchas formas». **Una misma orden, `h.golpear()`, y cada objeto responde con lo suyo:** la espada da un tajo, el arco dispara dos flechas, el bastón lanza un hechizo. Quien da la orden no pregunta qué herramienta tiene enfrente.

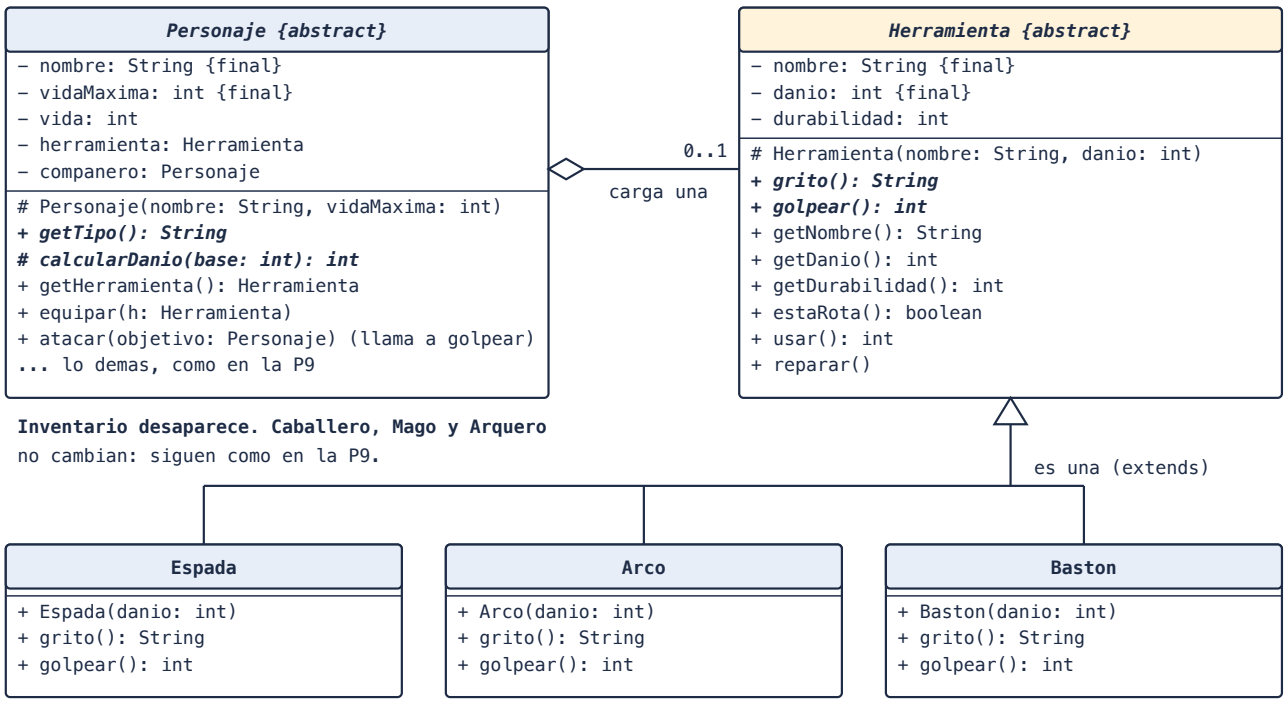
Para que eso funcione hacen falta tres ingredientes. Los tres aparecen en el código de hoy:

INGREDIENTE	QUÉ ES	EN ESTA PRÁCTICA
1 · La clase base dice qué	Una clase abstracta declara los métodos que toda hija debe tener, sin decir cómo: `public abstract String grito();`	`Herramienta`, con `grito()` y `golpear()`
2 · Cada hija dice cómo	Cada clase hija escribe esos métodos a su manera, con `@Override`	`Espada`, `Arco` y `Baston`: tres gritos y tres reglas de golpe
3 · El juego llama sin preguntar	Una variable del tipo base, `Herramienta h`, puede guardar cualquier hija. Al llamar `h.golpear()`, Java mira **el objeto**, no la variable, y corre el método de su clase	`Personaje.atacar` y el `for` de `Comparar`

Ya lo usaste en la P9 sin nombrarlo. `Personaje.atacar` llama a `calcularDanio(base)`, y según el objeto corre el del caballero (suma 5), el del mago (duplica) o el del arquero (suma 3). `atacar` no pregunta de qué tipo es el personaje: Java lo decide por el objeto. Hoy haces lo mismo, a propósito, con las herramientas.

El diagrama

Así queda la familia de herramientas. En cada clase aparece **solo lo que esa clase escribe**; lo heredado no se repite.



cursiva: abstracto, cada hija lo escribe con @Override. Amarillo: lo nuevo de esta practica.

Prepara tu proyecto

1. Copia tu carpeta de la P9 completa (o la `base-p09` del .zip) y llámala `p10`. Trabaja en la copia.
2. Del .zip, **reemplaza** tu `src/mazmorra/Main.java` por el nuevo y copia `Comparar.java` y `Moldes.java` junto a él.

```
p10/
  src/mazmorra/   # tus clases de la P9 con el Main nuevo, Comparar y
Moldes
  lib/  test/     # de la P9: no los toques ni los compiles
```

¿Arrancas de la base del .zip?

Antes de cambiar nada, compílala y córrela con el `Main` de la P9 para ver que funciona: Legolas debe quedar en 75. Luego reemplaza el `Main`.

```
javac -d out src/mazmorra/*.java
# compila las siete clases de la P9 y deja los .class en out
java -cp out mazmorra.Main
# corre el Main de la P9 buscando las clases en out
```

En Windows usa `\` en vez de `/` en las rutas.

Parte A · Ejemplo guiado: la clase base y su primera hija

Este ejemplo hace los tres ingredientes con **una** herramienta, el arco. Síguelo en orden y escríbelo en tu proyecto: cada paso trae el código completo y el concepto que aplica. En el paso A4 compruebas que funciona. En la Parte B repites el patrón con la espada y el bastón.

A1 · La clase base dice qué: `Herramienta` se vuelve abstracta

clase abstracta **método abstracto** **constructor protected**

En el juego ya no existe «una herramienta» a secas: toda herramienta es espada, arco o bastón. Por eso `Herramienta` se vuelve **abstracta**, como `Personaje` en la P9: sirve para agrupar a sus hijas, no para crear objetos. Así queda tu `Herramienta.java` completo; lo nuevo son la primera línea, el `protected` del constructor y los dos métodos abstractos:

```java
package mazmorra;

// Ya no existe "una herramienta" a secas: toda herramienta es una
// espada, un arco o un baston. Por eso la clase es abstracta.
public abstract class Herramienta {
    private final String nombre;
    private final int danio;
    private int durabilidad;

    protected Herramienta(String nombre, int danio) {
        this.nombre = nombre;
        this.danio = entre(danio, 1, 50);
        this.durabilidad = 100;
    }

    // 1. GRITO: cada herramienta suena distinto al atacar
    public abstract String grito();

    // 2. GOLPE: cada herramienta decide cuanto dano hace un golpe
    public abstract int golpear();

    public String getNombre() {
        return nombre;
    }

    public int getDanio() {
        return danio;
    }

    public int getDurabilidad() {
        return durabilidad;
    }

    public boolean estaRota() {
        return durabilidad == 0;
    }

    // rota no hace dano; si no, se desgasta un punto
```

```
    public int usar() {
        if (estaRota()) {
            return 0;
        }
        durabilidad--;
        return danio;
    }

    public void reparar() {
        durabilidad = 100;
    }

    private int entre(int valor, int min, int max) {
        return Math.max(min, Math.min(max, valor));
    }
}
```

- Un **método abstracto** no tiene cuerpo: termina en `;`. Dice «toda herramienta sabe gritar y golpear», pero no dice cómo. **Cada hija está obligada a escribirlo**; si no, no compila.
- `usar()` sigue igual que en la P9: desgasta un punto y devuelve el daño. `golpear()` es nuevo: cada hija decide cómo convierte un uso en un golpe.
- El constructor pasa a `protected` porque ya solo lo llaman las hijas, con `super(...)`.

Desde este momento **no se pueden crear objetos `Herramienta`**. Por eso el .zip trae un `Main` nuevo: el de la P9 hacía `new Herramienta("Espada", 20)`, y si lo compilas sale:

> **EL MAIN DE LA P9 YA NO COMPILA**
>
> ```
> Main.java:5: error: Herramienta is abstract; cannot be instantiated
> Main.java:6: error: Herramienta is abstract; cannot be instantiated
> Main.java:7: error: Herramienta is abstract; cannot be instantiated
> ```

A2 · La hija dice cómo: el `Arco`

herencia (extends) · super(...) · sobrescritura (@Override)

Crea `src/mazmorra/Arco.java`:

```
package mazmorra;

public class Arco extends Herramienta {
    public Arco(int danio) {
        super("Arco", danio);
    }

    @Override
    public String grito() {
        return "dispara dos flechas";
    }

    // solo el arco tiene flechas: Herramienta no sabe de esto
    public int getFlechas() {
        return 2;
    }

    // dos flechas: el arco se usa dos veces en un solo golpe
    @Override
    public int golpear() {
        return usar() + usar();
    }
}
```

- **El constructor** recibe solo el daño y le pasa a `super` el nombre `"Arco"`. El daño se ajusta de 1 a 50 en `Herramienta`, como en la P9.
- **Los dos `@Override`** son la respuesta del arco a las dos obligaciones: su grito y su regla de golpe. El arco dispara dos flechas: usa el arco dos veces en un solo golpe, así que pega el doble y se desgasta dos puntos.
- `getFlechas()` es un método **que solo tiene el arco**: `Herramienta` no sabe de flechas. Lo vas a necesitar en la Parte D, donde se ve cómo llegar a él desde una variable `Herramienta`.
- `usar()`, `getDanio()` y lo demás no aparecen: el arco los hereda tal cual.

> Si a una hija le falta uno de los dos métodos, no compila. Por ejemplo, sin `grito()`:
>
> > SALE
> >
> > ```
> > Arco.java:3: error: Arco is not abstract and does not override abstract method grito() in Herramienta
> > ```
>
> Y si lo escribes sin `public`, sale `attempting to assign weaker access privileges; was public`: una hija no puede cerrar lo que la madre dejó abierto.

A3 · El juego llama sin preguntar: el personaje carga una herramienta

agregación **referencia de la clase base** **enlace dinámico**

En la parte 2 **el inventario desaparece**: cada personaje carga una sola herramienta, la de su tipo. Borra `Inventario.java` y, en `Personaje`, cambia el atributo `inventario` por este y quita el `new Inventario()` del constructor:

```
private Herramienta herramienta;       // agregacion: se forja afuera
```

Luego cambia `getCuantasHerramientas()` y `getHerramienta(i)` por un solo `getHerramienta()`, y deja `equipar` y `atacar` así (lo demás de `atacar` no cambia):

```
    public Herramienta getHerramienta() {
        return herramienta;
    }

    // una sola herramienta: la nueva reemplaza a la anterior
    public void equipar(Herramienta h) {
        this.herramienta = h;
    }

    public void atacar(Personaje objetivo) {
        if (!estaVivo() || herramienta == null) {
            return;
        }
        int base = herramienta.golpear();   // P10: la herramienta
decide
        if (base > 0) {
            objetivo.recibirDanio(calcularDanio(base));
        }
    }
```

- La herramienta se forja afuera y llega por `equipar`: es **agregación**, como en la P9. Si equipas otra, reemplaza a la anterior.
- El atributo `herramienta` es una **variable de tipo `Herramienta`**: el personaje **no sabe** si carga un arco o una espada. Cuando llama a `herramienta.golpear()`, Java mira el **objeto** y corre el `golpear` de su clase. Eso es el **enlace dinámico**: el método se decide por el objeto, no por el tipo de la variable.

> Si se te olvida quitar algo del inventario, sale `cannot find symbol` sobre `Inventario`:
>
> > SALE
> >
> > ```
> > Personaje.java:7: error: cannot find symbol
> > Personaje.java:14: error: cannot find symbol
> > ```

A4 · Punto de control: corre el `Main`

Con `Herramienta`, `Arco` y el cambio a `Personaje`, el juego ya compila. Este es el `Main` del .zip: Legolas, el arquero, carga su arco (los otros dos personajes reciben lo suyo en la Parte C):

```
package mazmorra;

public class Main {
    public static void main(String[] args) {
        Herramienta arco = new Arco(15);  // tipo Herramienta, objeto
Arco
        Personaje legolas = new Arquero("Legolas");
        Personaje conan = new Caballero("Conan");
        legolas.equipar(arco);

        System.out.println("Legolas " + arco.grito());
        legolas.atacar(conan);
        System.out.println(conan);
        System.out.println(arco.getNombre() + ": durabilidad "
                + arco.getDurabilidad());
    }
}
```

```
javac -d out src/mazmorra/*.java
# compila el juego y deja los .class en out
java -cp out mazmorra.Main
# corre el Main buscando las clases en out
```

> **DEBE SALIR**
>
> ```
> Legolas dispara dos flechas
> Conan (Caballero) 87/120
> Arco: durabilidad 98
> ```

Sigue el ataque de Legolas: `atacar` llama a `herramienta.golpear()`; el objeto es un `Arco`, así que corren dos flechas de 15: **30**, y el arco baja a 98. Luego `calcularDanio(30)`: Legolas es `Arquero`, suma 3. Conan recibe **33** y queda en 87. Dos decisiones, la de la herramienta y la del personaje, y ningún `if` que pregunte qué es cada uno.

> Si Conan te queda en 102 y el arco en 99, tu `atacar` llama a `usar()` en vez de `golpear()`: vuelve al paso A3.
>
> **El patrón que acabas de seguir**
>
> 1. **La hija:** `extends Herramienta` y un constructor que llama a `super("Nombre", danio)`.
> 2. **Su grito:** `grito()` con `@Override`.
> 3. **Su regla de golpe:** `golpear()` con `@Override`, usando `usar()` o `getDanio()`.

Parte B · Por tu cuenta: la espada y el bastón

Crea `Espada.java` (la del caballero) y `Baston.java` (el del mago) en `src/mazmorra` con **el mismo patrón que el arco**. Solo cambian el nombre que va en `super(...)`, el grito y la regla de golpe. Esta es la tarjeta de cada herramienta:

CLASE	`SUPER(...)`	`GRITO()` DEVUELVE	`GOLPEAR()`: SU REGLA
`Arco` (hecho en la Parte A)	`"Arco"`	`"dispara dos flechas"`	solo el arco tiene flechas: Herramienta no sabe de esto: `return usar() + usar();`
`Espada`	`"Espada"`	`"lanza un tajo"`	un tajo: todo el dano, y la espada se desgasta: `return usar();`
`Baston`	`"Baston"`	`"lanza un hechizo"`	un hechizo: todo el dano, y la magia no desgasta el baston: `return getDanio();`

Compila después de cada clase con `javac -d out src/mazmorra/*.java`. El `Main` tiene que seguir dando lo mismo que en A4. Comprobarás las tres en la Parte C.

Parte C · Sin y con polimorfismo

referencia de la clase base **instancias de las clases derivadas** **enlace dinámico**

Este es el momento de la práctica. Vas a ver el mismo trabajo hecho de dos maneras: una que pregunta qué es cada herramienta y otra que no pregunta nada.

C1 · El archivo completo

Este es `Comparar.java` tal como viene en el .zip. Léelo completo antes de escribir:

```java
package mazmorra;

// El mismo recorrido dos veces: primero preguntando que es cada
// herramienta, luego sin preguntar.
public class Comparar {
    public static void main(String[] args) {
        System.out.println("-- Sin polimorfismo --");
        sinPolimorfismo(arsenal());
        System.out.println("-- Con polimorfismo --");
        conPolimorfismo(arsenal());
    }

    // un arreglo del tipo base con objetos de tres clases distintas
    static Herramienta[] arsenal() {
        return new Herramienta[] {
            new Espada(20), new Arco(15), new Baston(10)
        };
    }

    // el juego pregunta que herramienta es cada una y decide por ella
    static void sinPolimorfismo(Herramienta[] arsenal) {
        for (Herramienta h : arsenal) {
            String grito;
            int golpe;
            if (h instanceof Espada) {
                grito = "lanza un tajo";
                golpe = h.usar();
            } else if (h instanceof Arco) {
                grito = "dispara dos flechas";
                golpe = h.usar() + h.usar();
            } else if (h instanceof Baston) {
                grito = "lanza un hechizo";
                golpe = h.getDanio();
            } else {
                grito = "no se que es esto";
                golpe = 0;
            }
            imprimir(h, grito, golpe);
```

```
        }
    }

    // el juego solo da la orden; cada herramienta responde con lo suyo
    static void conPolimorfismo(Herramienta[] arsenal) {
        // TU CODIGO: un for-each sobre arsenal que, por cada h, llame a
        // imprimir(h, h.grito(), h.golpear()). Sin ningun instanceof.
    }

    static void imprimir(Herramienta h, String grito, int golpe) {
        System.out.println(h.getNombre() + " " + grito + ": golpe de "
                + golpe + ", durabilidad " + h.getDurabilidad());
    }
}
```

Tiene cinco partes:

PARTE	QUÉ HACE
`main`	Corre el recorrido dos veces: primero sin polimorfismo y luego con polimorfismo.
`arsenal()`	Crea **un arreglo del tipo base**, `Herramienta[]`, con objetos de **tres clases distintas**: una espada, un arco y un bastón. Es justo lo que pide el manual: una referencia de la clase más general e instancias de las derivadas.
`sinPolimorfismo`	**Ya viene hecho.** Para cada herramienta pregunta qué es con `instanceof` (3 preguntas) y decide por ella qué grito y qué golpe le tocan. Si no la reconoce, cae en «no se que es esto».
`conPolimorfismo`	**Lo escribes tú.** Hace lo mismo, pero sin preguntar.
`imprimir`	Escribe un renglón con el nombre, el grito, el golpe y la durabilidad. Las dos mitades lo usan.

C2 · Tu parte: la mitad con polimorfismo

Dentro de `conPolimorfismo`, donde dice `TU CODIGO`, escribe un `for-each` sobre `arsenal` que, por cada herramienta `h`, llame a `imprimir(h, h.grito(), h.golpear())`. **Sin un solo `instanceof`**. Luego compila y corre:

```
javac -d out src/mazmorra/*.java
# compila el juego con las tres herramientas y Comparar
java -cp out mazmorra.Comparar
# corre las dos mitades
```

```
DEBE SALIR
-- Sin polimorfismo --
Espada lanza un tajo: golpe de 20, durabilidad 99
Arco dispara dos flechas: golpe de 30, durabilidad 98
Baston lanza un hechizo: golpe de 10, durabilidad 100
-- Con polimorfismo --
Espada lanza un tajo: golpe de 20, durabilidad 99
Arco dispara dos flechas: golpe de 30, durabilidad 98
Baston lanza un hechizo: golpe de 10, durabilidad 100
```

Así queda tu método cuando está bien:

```
    // el juego solo da la orden; cada herramienta responde con lo suyo
    static void conPolimorfismo(Herramienta[] arsenal) {
        for (Herramienta h : arsenal) {
            imprimir(h, h.grito(), h.golpear());
        }
    }
```

C3 · Qué acabas de ver

Las dos mitades imprimen exactamente lo mismo. La diferencia está en el código: la primera tiene que conocer todas las herramientas y decidir por ellas; la segunda no conoce ninguna. Solo da la orden, `h.grito()` y `h.golpear()`, y Java corre el método de la clase de cada objeto.

Piensa que mañana llega un hacha. En la mitad que pregunta hay que abrir `Comparar` y agregar otro `else if`; si alguien lo olvida, el hacha cae en «no se que es esto». En la mitad con polimorfismo **no se toca nada**: basta escribir la clase `Hacha` y ponerla en el arreglo. Cada objeto trae sus respuestas consigo.

Parte D · Los moldes

upcast instanceof downcast

D1 · La teoría: dos tipos en cada línea

En `Herramienta h = new Arco(15);` hay **dos tipos**, y no son lo mismo:

	EL TIPO DE LA VARIABLE	EL TIPO DEL OBJETO
En el ejemplo	`Herramienta`, lo que va a la izquierda	`Arco`, lo que va después del `new`
Quién lo mira	el **compilador**, al hacer `javac`	**Java**, al correr el programa
Qué decide	**qué métodos puedes llamar**: solo los que tiene `Herramienta`	**cuál versión corre**: la del arco
¿Cambia?	no, se fija al declararla	la variable puede apuntar después a otro objeto

Por eso `h.grito()` compila (toda herramienta grita) y corre el grito del arco. Pero `h.getFlechas()` **no compila**, aunque el objeto sí sea un arco: el compilador solo ve una `Herramienta`, y no toda herramienta tiene flechas.

A cambiar el tipo de la variable se le llama **cast** o **moldeo**. Hay dos direcciones:

	UPCAST: HACIA ARRIBA	DOWNCAST: HACIA ABAJO
Va de	la hija a la base: `Arco` → `Herramienta`	la base a la hija: `Herramienta` → `Arco`
Cómo se escribe	solo, sin nada: `Herramienta h = new Arco(15);`	con el tipo entre paréntesis: `Arco a = (Arco) h;`
¿Puede fallar?	nunca: un arco **siempre es** una herramienta	sí: una herramienta **puede no ser** un arco
Para qué sirve	tratar a todas igual, como en el arreglo de la Parte C	usar lo que solo tiene la hija, como `getFlechas()`

Como el downcast puede fallar, antes se pregunta con **`instanceof`**: `h instanceof Arco` da `true` si el objeto es un arco (o una hija del arco) y `false` si no. Pregunta por el **objeto**, no por la variable.

D2 · Los moldes en código

Para este ejemplo, el `Arco` tiene un método que solo él tiene, `getFlechas()`: cuántas flechas dispara. Está en tu `Arco` del paso A2. Este es `Moldes.java`:

```
package mazmorra;

// La variable es de un tipo; el objeto puede ser de ese tipo o de un
hijo.
public class Moldes {
    public static void main(String[] args) {
        Herramienta h = new Arco(15);           // 1 upcast: automatico
        System.out.println("1 " + h.grito());   //   manda el objeto

        // h.getFlechas();                      // 2 no compila

        System.out.println("3 " + (h instanceof Arco));
        System.out.println("3 " + (h instanceof Espada));

        Arco a = (Arco) h;                      // 4 downcast: se escribe
        System.out.println("4 flechas: " + a.getFlechas());

        // Espada e = (Espada) h;               // 5 compila, pero truena
    }
}
```

```
java -cp out mazmorra.Moldes
# corre los moldes (ya está compilado con lo demás)
```

> **DEBE SALIR**
>
> ```
> 1 dispara dos flechas
> 3 true
> 3 false
> 4 flechas: 2
> ```

- **1, upcast:** la variable es `Herramienta` y el objeto es un `Arco`. El grito es el del arco: manda el objeto.
- **3, `instanceof`:** el objeto es un arco (`true`), no una espada (`false`).
- **4, downcast:** `(Arco) h` da una variable de tipo `Arco`, y con ella sí puedes llamar a `getFlechas()`. Funciona porque el objeto sí es un arco.

D3 · Los dos errores

Quita el `//` de la **línea 2** y compila. **No compila**: la variable es `Herramienta`, y `Herramienta` no tiene `getFlechas()`. El compilador no mira el objeto:

```
SALE
Moldes.java:9: error: cannot find symbol
symbol:   method getFlechas()
location: variable h of type Herramienta
```

Vuelve a poner el `//`. Ahora quítalo de la **línea 5**, compila y corre. Esta vez **sí compila**, porque una herramienta *podría* ser una espada, pero **truena al correr**: Java mira el objeto, ve un arco, y un arco no se puede convertir en espada:

```
SALE
Exception in thread "main" java.lang.ClassCastException: class mazmorra.Arco cannot be cast to
class mazmorra.Espada (mazmorra.Arco and mazmorra.Espada are in unnamed module of loader 'app')
```

Vuelve a poner el `//`. Resumen: **el compilador revisa el tipo de la variable y Java revisa el tipo del objeto.** Por eso un downcast va siempre después de un `instanceof`. Y si te descubres escribiendo `instanceof` y cast para decidir qué hacer, como en la mitad sin polimorfismo, casi siempre lo que falta es un método en la clase base.

Si terminas: retos

- **El hacha:** escribe `Hacha` con su grito y su regla de golpe (inventa una) y agrégala al arreglo de `Comparar`. Corre y mira qué dice cada mitad. Luego arregla la mitad que pregunta.
- **Una ronda completa:** en el `Main`, dale su espada a Conan y su bastón a un mago, y haz una ronda como en la P9. ¿Qué durabilidad tiene cada herramienta al final?

Si algo truena

No compila

- `Herramienta is abstract; cannot be instantiated`: queda algún `new Herramienta(...)`. ¿Reemplazaste el `Main`? ¿Estás compilando `test/`? No lo compiles.
- `... is not abstract and does not override abstract method ... in Herramienta`: a esa hija le falta `grito()` o `golpear()`.
- `attempting to assign weaker access privileges; was public`: escribiste un método sobrescrito sin `public`.
- `cannot find symbol: method golpear()`: en `Herramienta` falta el método abstracto, o lo escribiste con otro nombre.
- `cannot find symbol` sobre `Inventario` o `inventario`: quedó algo del inventario en `Personaje`; repasa el paso A3.

Compila, pero algo sale distinto

- **Conan queda en 102 y el arco en 99**: `atacar` llama a `usar()` en vez de `golpear()`.
- **La mitad «con» de `Comparar` sale vacía**: todavía no escribes el `for`.
- **Las dos mitades salen distintas**: tu grito o tu regla de golpe no son los de la tarjeta. La mitad que pregunta los tiene bien; compárala con tu clase.

La entrega

QUÉ	DETALLE
Cuándo	**Viernes 6 de noviembre, 23:59.** Sin IA, como siempre.
Con quién	En pareja (o solo). **Cada integrante sube todo en su propia carpeta.**

Dónde	Tu rama `entregas_apellido_nombre`, carpeta `entregas/apellido_nombre/p10/`. Solo push.
Qué	Tu carpeta `src` y una **bitácora en PDF**

La bitácora lleva: los integrantes (nombre y número de cuenta); capturas del `Main`, de `Comparar` y de `Moldes` (también con la línea 2 y la 5 probadas); y estas tres respuestas con tus palabras:

1. **Una orden, muchas respuestas:** en `Personaje`, ¿qué línea manda a llamar a `golpear`? Cuando Legolas le quita 33 a Conan, ¿qué clase decidió las dos flechas y cuál el +3? ¿Lo decidió el tipo de la variable o el del objeto?
2. **Sin y con polimorfismo:** si mañana llega un hacha, ¿qué tendrías que cambiar en cada mitad de `Comparar`? ¿Qué imprime la mitad que pregunta si nadie la actualiza?
3. **Los moldes:** copia el error de la línea 2 y lo que truena en la línea 5. Explica cada uno con el tipo de la variable y el del objeto: ¿quién encontró el problema, el compilador o Java al correr?

Cómo se califica, sobre 10:

QUÉ REVISO	PUNTOS
Fiel al diagrama: `Herramienta` abstracta con sus dos métodos abstractos; `Espada`, `Arco` y `Baston` con `extends`, `super(...)`, `@Override` y la regla de su tarjeta; `Personaje` con una sola herramienta y `atacar` llamando a `golpear()` (paso A3)	3.5
Corre: el `Main` y `Comparar` dan la salida de la guía, y tu mitad «con» es un `for` sin ningún `instanceof`	3
Moldes: corre y la línea 2 y la 5 están probadas	1
Bitácora: las capturas y las tres respuestas	2.5

Cada comando de abajo, con lo que hace:

```
git checkout entregas_apellido_nombre
# te pasa a tu rama de entregas
git pull origin main
# trae lo nuevo del repositorio del grupo
mkdir -p entregas/apellido_nombre/p10
# crea tu carpeta de esta práctica; copia ahí src y la bitácora en PDF
git add entregas/apellido_nombre/p10
# marca lo que vas a entregar
git commit -m "P10: polimorfismo, primera parte"
# lo guarda con un mensaje
git push
# lo sube: esto es la entrega
```

Guarda tu proyecto completo: la P11 (miércoles 11 de noviembre, interfaces) sigue sobre este código. Si los `.class` de `out` se cuelan, no pasa nada. El código tiene que ser suyo; si algo no sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.

Modelos de Programación Orientada a Objetos 2027-1 · Grupo 6 · Facultad de Ingeniería, UNAM
Prof. Oscar Manuel Ruiz Hurtado · Práctica 10 · Polimorfismo, primera parte (MADO-21, guía 10: jerarquía de clases, referencia de la clase base más general, instancias de las clases derivadas, cast e `instanceof`, polimorfismo en métodos) · noviembre de 2026